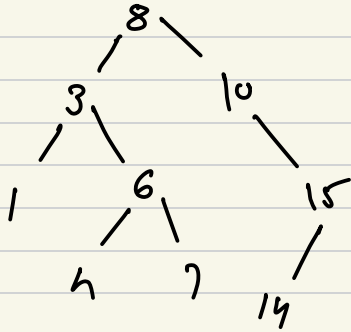


Tree Traversal



Preorder : VLR

↳ 8, 3, 1, 6, 4, 7, 10, 15, 14

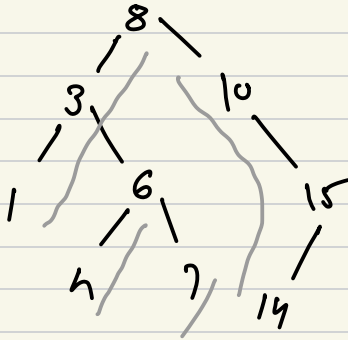
Inorder : LVR

↳ 1, 3, 4, 6, 7, 8, 10, 14, 15

Postorder : LRV

↳ 1, 4, 7, 6, 3, 14, 15, 10, 8

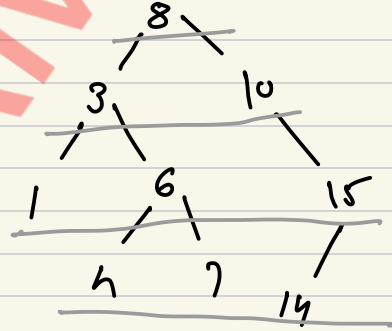
DFS (Stack)



14
15
4
7
1
6
5
10
8

Traversal : 8, 3, 1, 6, 4, 7, 10, 15, 14

BFS (Queue)

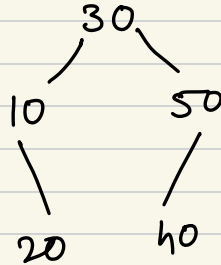


14	15	10	1	6	5	4	7	11		
---------------	---------------	---------------	--------------	--------------	--------------	--------------	--------------	---------------	--	--

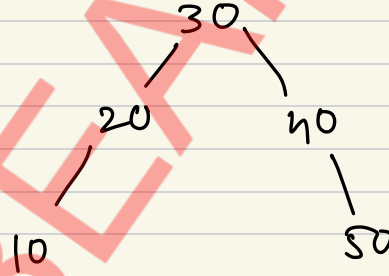
Traversal: 8, 3, 10, 1, 6, 15, 4, 7, 14
(level order traversal)

Skewed BST

30, 10, 50, 40, 20



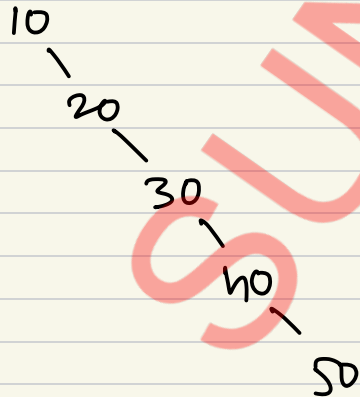
30, 20, 40, 50, 10



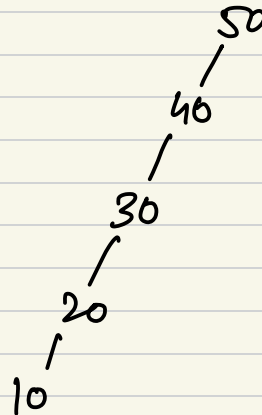
$$h \approx \log n$$

$$T(n) = O(\log n)$$

10, 20, 30, 40, 50



50, 40, 30, 20, 10



$$h \approx n$$

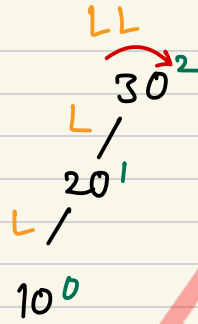
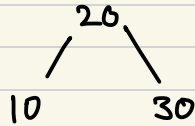
$$T(n) = O(n)$$

(Skewed BSTs)

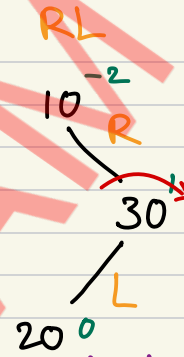
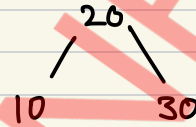
Imbalance cases



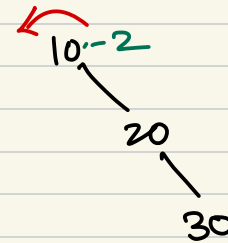
Left rotation



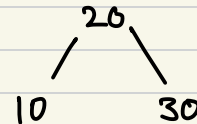
Right rotation



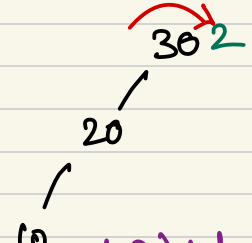
Right rotation



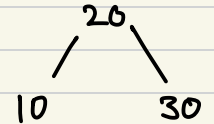
Left rotation



Left rotation



Right rotation

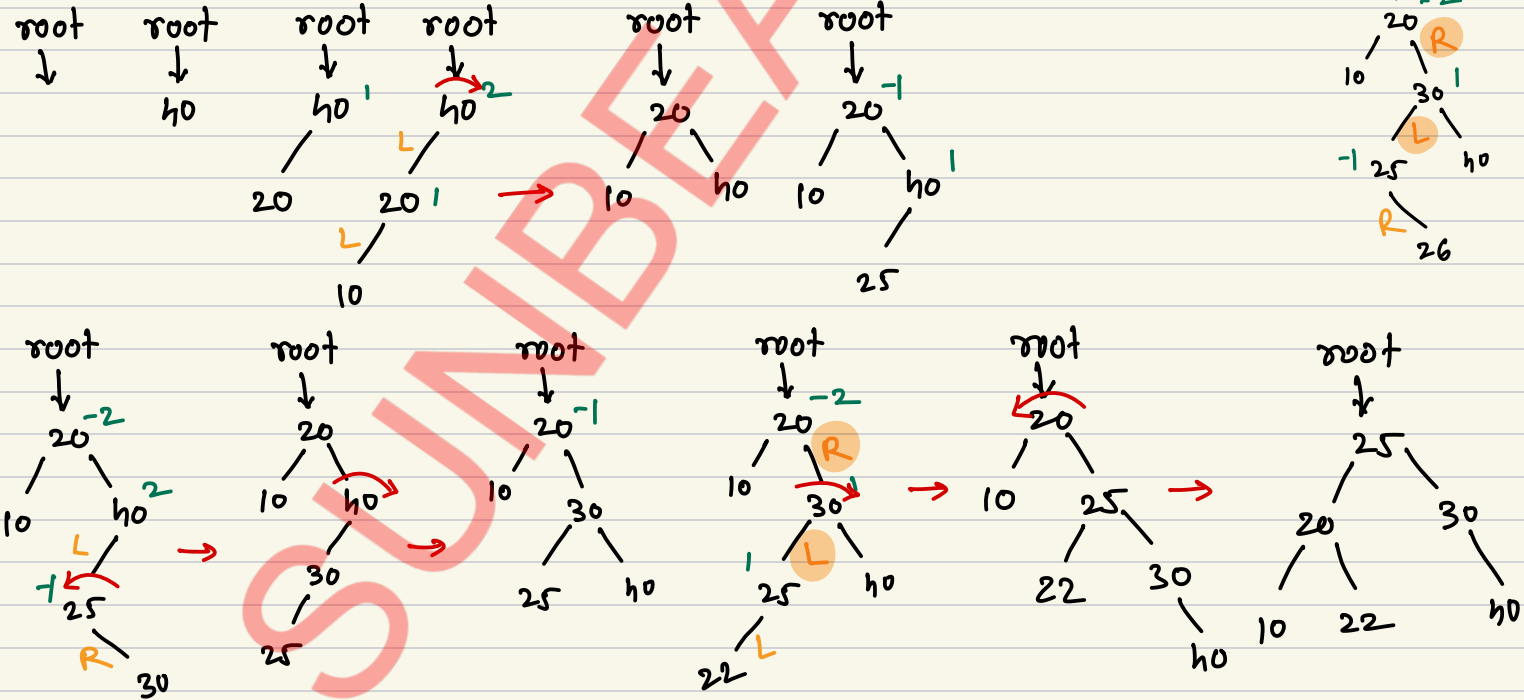


Single Rotation

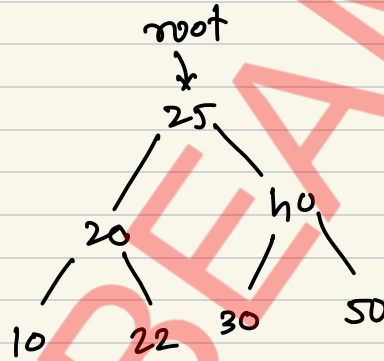
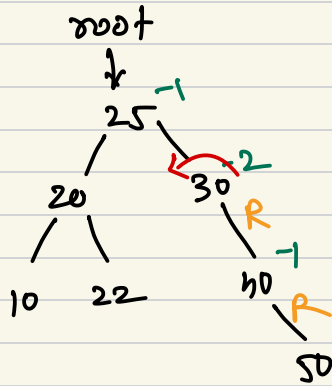
Double Rotation

AVL Tree - Add

Keys : 40, 20, 10, 25, 30, 22, 50



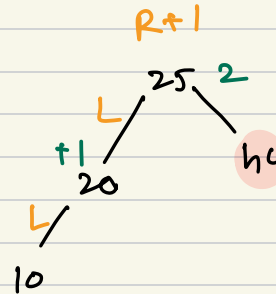
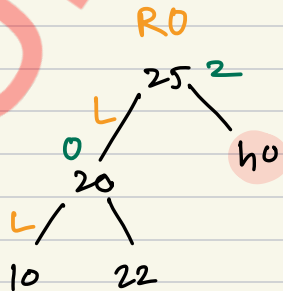
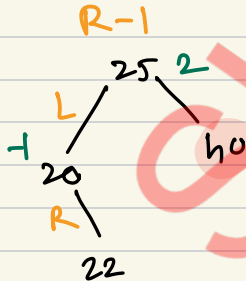
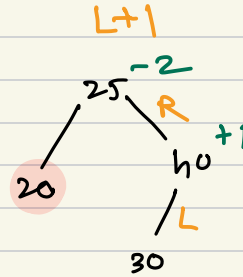
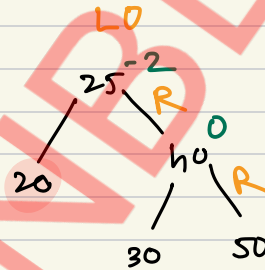
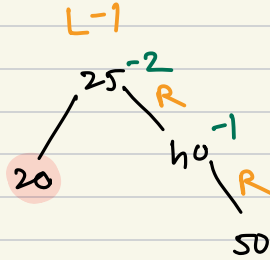
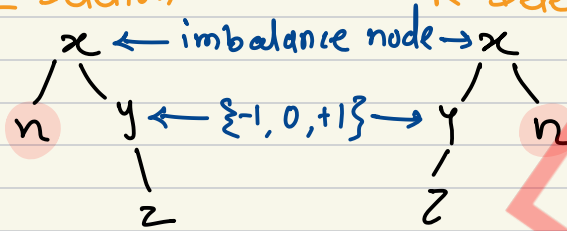
AVL Tree - Add



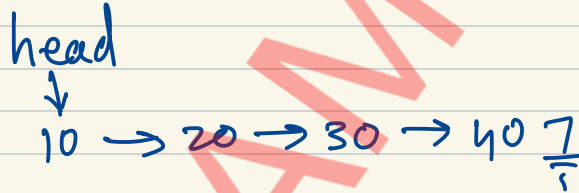
AVL Tree - Delete

L-Deletion

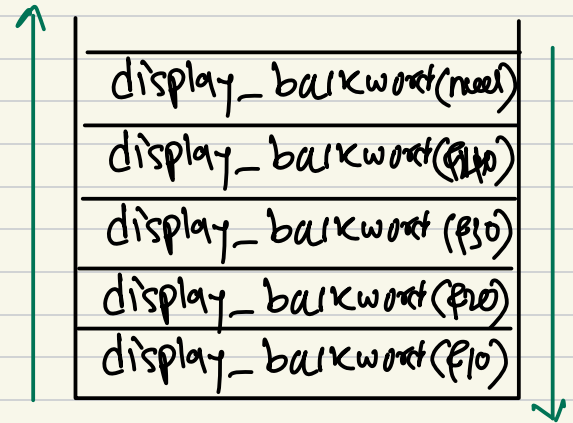
R-Deletion



Head / Non-tail Recursion

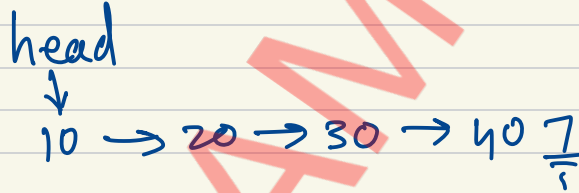


```
void display_backward(node_t *trav)
{
    if (trav == null)
        return;
    display_backward(trav → next);
    printf("%d", trav → data);
}
```

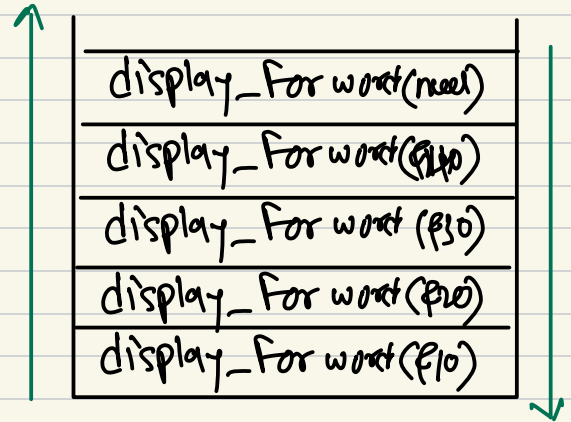


Output : 40 , 30 , 20 , 10

Tail Recursion

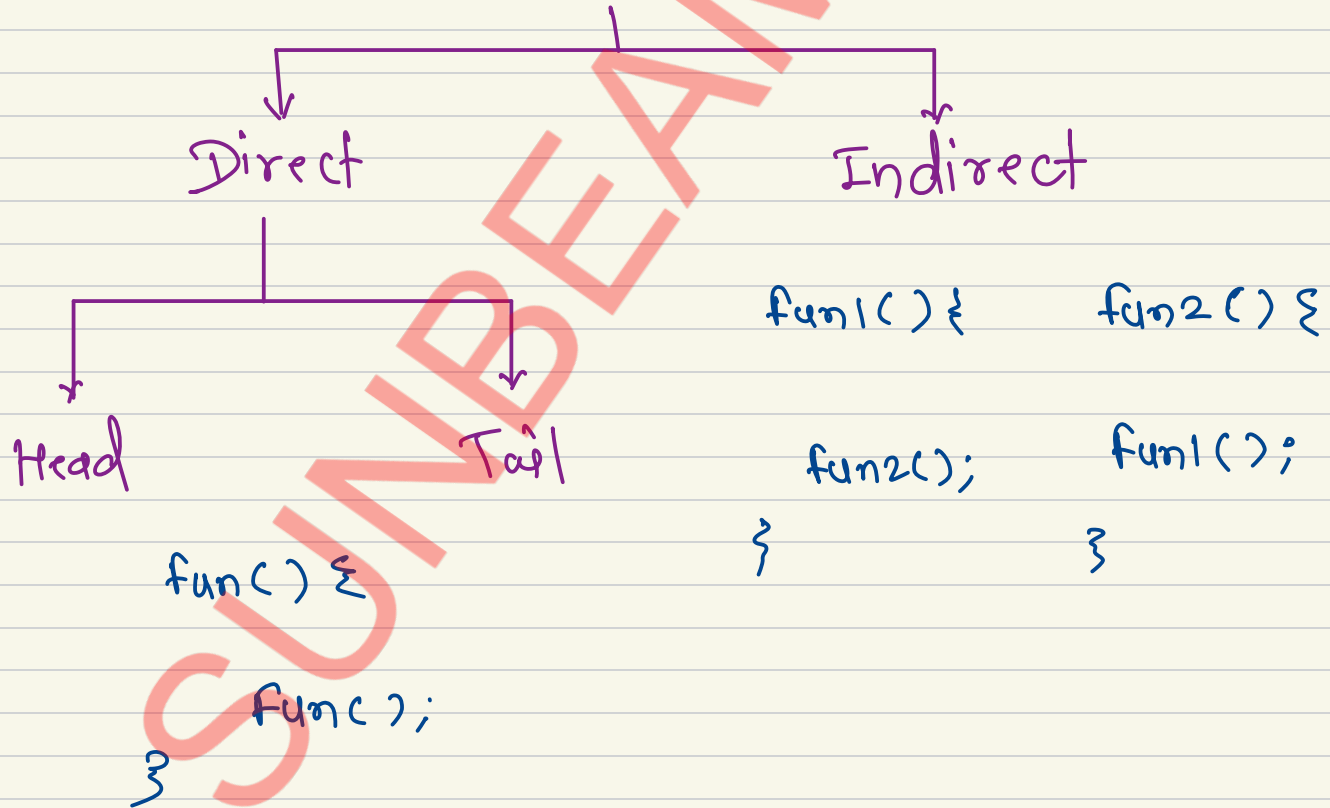


```
void display_forward(node_t *trav)
{
    if(trav == null)
        return;
    printf("%d", trav->data);
    display_forward(trav->next);
}
```



Output : 10, 20, 30, 40

Recursion



Reverse List using recursion

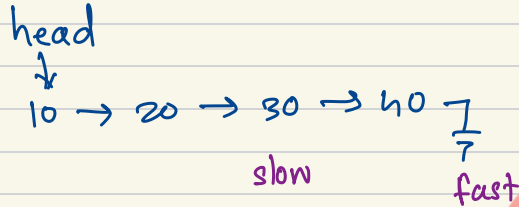
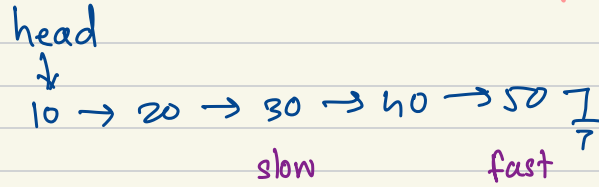
head
↓
10 → 20 → 30 → 40 → NULL

head
↓
40 → 30 → 20 → 10 → NULL

```
node_t *reverse_list(node_t *curr) {  
    if (curr->next == NULL) {  
        head = curr;  
        return curr;  
    }  
    last = reverse_list(curr->next);  
    last->next = curr;  
    curr->next = NULL;  
    return curr;  
}
```

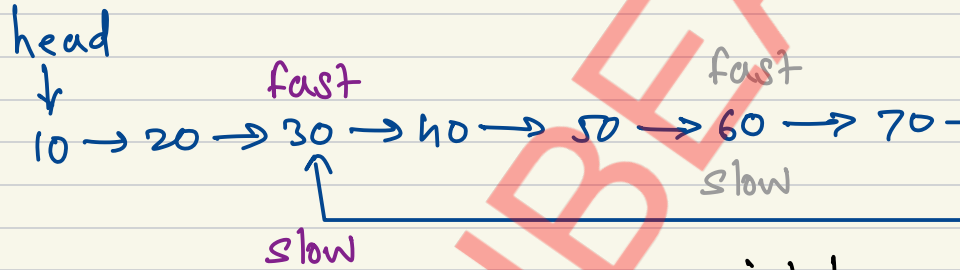
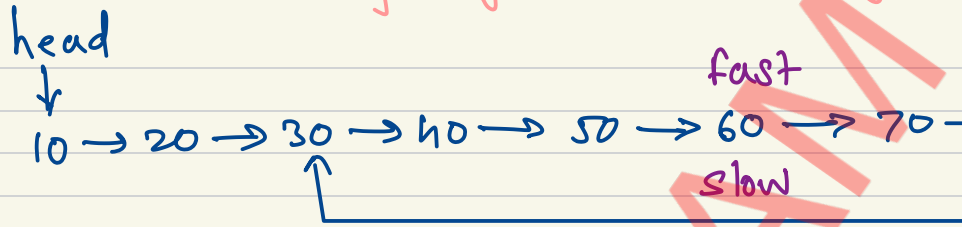
	curr	last
reverse_list(10)	10	20
reverse_list(20)	20	30
reverse_list(30)	30	40
reverse_list(40)	40	

Middle node of a list



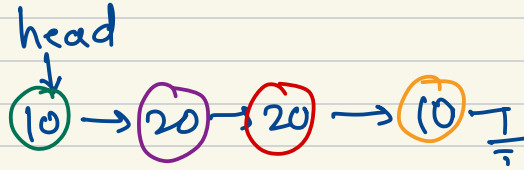
```
node_t * find_mid(node_t * head)
{
    node_t * fast = head;
    node_t * slow = head;
    while (fast != NULL && fast->next != NULL)
    {
        fast = fast->next->next;
        slow = slow->next;
    }
    return slow;
}
```

Floyd Cyclic Detection Algorithm

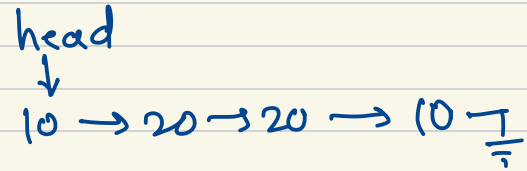


```
int has_cycle (node_t * head)
{
    node_t * fast = head;
    node_t * slow = head;
    while (fast != NULL && fast->next != NULL)
    {
        fast = fast->next->next;
        slow = slow->next;
        if (fast == slow)
            return 1;
    }
    return 0;
}
```

List palindrome or not?



$$\begin{array}{l} \text{to push} = n \\ \text{to check} = n \\ \hline 2n \end{array}$$



$$\begin{array}{l} \text{to push} = n \\ \text{to check} = n/2 \\ \hline \frac{3}{2}n \end{array}$$